The Journal of Open Source Software

visualCaseGen: An SMT-based Experiment Configurator for Community Earth System Model

Alper Altuntas [1], **Isla R. Simpson** [1], **Scott D. Bachman** [1], **Manish Venumuddula** [1], **Samuel Levis** [1], **Brian Dobbins** [1], **William J. Sacks** [1], **and Gokhan Danabasoglu** [1]

1 U.S. National Science Foundation National Center for Atmospheric Research (NSF NCAR), Boulder, CO

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 01 July 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Introduction

visualCaseGen is a graphical user interface (GUI) designed to streamline the creation and configuration of Community Earth System Model (CESM) experiments. CESM is a state-of-the-art climate model that allows researchers to simulate the Earth system across a wide range of spatial and temporal scales and at varying levels of complexity, depending on scientific objectives (Danabasoglu et al., 2020). However, configuring non-standard or idealized CESM experiments is often a technically demanding and time-consuming process, requiring detailed knowledge of component compatibility, grid definitions, parameterization schemes, and model hierarchies. visualCaseGen simplifies this process by guiding users through each stage of experiment setup and automating key configuration steps via an intuitive, interactive interface.

To ensure consistency and compatibility across various model settings, visualCaseGen incorporates a constraint solver based on satisfiability modulo theories (SMT) (De Moura & Bjørner, 2011). This solver systematically analyzes dependencies between configuration parameters, detects conflicts, and provides detailed explanations of incompatibilities, allowing users to make informed adjustments. The SMT-based approach enables dynamic, real-time validation of model settings, significantly reducing setup errors and ensuring that only scientifically viable configurations are selected. Complementing the solver, the visualCaseGen backend implements a *Stage Mechanism* that divides the workflow into structured, sequential steps. This mechanism works in tandem with the solver to manage variable precedence, guide user input, and enforce constraints efficiently.

On the frontend, visualCaseGen is implemented as a Jupyter-based GUI (Jupyter, 2015), offering an intuitive, step-by-step interface for browsing standard CESM configurations, defining custom experiment setups, and modifying grid and component settings. Designed with a wizard-like interface, visualCaseGen walks users through each stage of the CESM configuration process, ensuring that all necessary settings are selected in a logical sequence while dynamically updating available options based on user choices. Additionally, the tool features utilities for creating and editing model input files, such as ocean grid, bathymetry, and land surface properties, further simplifying model customization and eliminating the need for manual file modifications.

By automating and simplifying CESM configuration, visualCaseGen makes the model more accessible and custumizable, particularly for researchers exploring hierarchical modeling (Maher et al., 2019), idealized experiments (Polvani et al., 2017), or custom coupled simulations. As such, the tool allows users to focus on their scientific objectives rather than technical setup challenges, and ultimately enables a more efficient and streamlined experiment workflow.

The Journal of Open Source Software

Statement of need

CESM is a highly flexible and comprehensive modeling framework that encompasses multiple components representing the atmosphere, ocean, land, ice, river, biogeochemistry, and other systems. While this flexibility supports a wide range of scientific experiments, it also introduces significant complexity in model configuration. Setting up custom CESM experiments requires navigating intricate component compatibility constraints, grid configurations, and parameterization choices, often demanding extensive expertise in the model's internal structure. For non-standard experiments, users must manually modify CESM's codebase and runtime parameter files, maintain numerical and scientific consistency, and troubleshoot compatibility issues. This process is time-intensive, error-prone, and can take weeks before yielding a functional setup.

A recent study by Wu et al. (2021) exemplifies these challenges. Their work involved configuring an idealized CESM experiment to study atmosphere-ocean interactions using two simplified aquaplanet models: one without continents and another with a pole-to-pole strip continent. The goal was to investigate how these configurations influence Hadley circulation, equatorial upwelling, and precipitation patterns (Figure 1). However, setting up these experiments required extensive manual intervention, including modifying CESM codebase, creating custom input files, adjusting runtime parameters, consulting domain experts for component-specific configurations, and conducting numerous trial-and-error iterations. visualCaseGen was developed to address such usability barriers and streamline CESM experiment setup. As an interactive GUI, it eliminates the need for manual modifications and provides an intuitive, structured workflow for constructing model configurations.

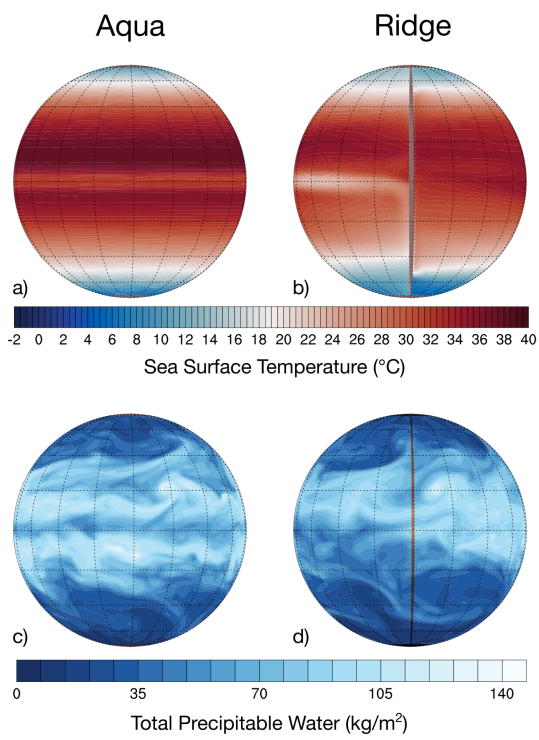

Figure 1: Sea surface temperature and precipitable water distribution from Aqua and Ridge planet simulations using CESM. Adapted from Wu et al. (2021).

DRAFT

Constraint Solver

One of the main challenges in configuring CESM experiments is ensuring that different model settings remain compatible. CESM's configuration involves determining components, physics, grids, and parameterization choices, many of which have strict compatibility constraints. visualCaseGen addresses this challenge by integrating an SMT-based constraint solver, built using the Z3 solver (De Moura & Bjørner, 2008). Z3 was chosen for its robust Python API and its efficiency in managing complex logical relationships involving multiple parameter types such as integers, reals, booleans, and strings. As such, Z3 is well suited to handling the intricate dependencies and constraints inherent in CESM configurations.

In visualCaseGen, constraints are specified as key-value pairs, where the key represents a Z3 logical expression defining a condition, and the value is the error message displayed when the constraint is violated. These constraints enforce compatibility rules and prevent invalid model configurations. Below are three examples of visualCaseGen constraints with increasing complexity, demonstrating how the SMT solver can enforce simple value bounds, conditional dependencies, and more complex multi-component logical rules:

```
LND_DOM_PFT >= 0.0:
    "PFT/CFT must be set to a nonnegative number",

Implies(OCN_GRID_EXTENT=="Regional", OCN_CYCLIC_X=="False"):
    "Regional ocean domain cannot be reentrant"
    "(due to an ESMF limitation.)",

Implies(And(COMP_OCN=="mom", COMP_LND=="slnd", COMP_ICE=="sice"),
        OCN_LENY<180.0):
    "If LND and ICE are stub, custom MOM6 grid must exclude poles"
    "to avoid singularities in open water",
```

Why Use a Constraint Solver?

Configuring CESM is inherently a constraint satisfaction problem (CSP), which can quickly become computationally complex as the number of configuration variables increases (Biere et al., 2009). Manually enforcing constraints would be impractical, making an SMT solver an ideal choice. The benefits of using a solver include:

- **Detecting Hidden Conflicts:** Individual constraints may be satisfied independently, yet their combination can lead to conflicts that are nontrivial to detect manually.
- **Preventing Dead-Ends:** Without a solver, users may unknowingly select settings that lead to an unsatisfiable configuration, forcing them to restart their setup. Thanks to the solver, visualCaseGen dynamically guides users toward valid options.
- **Enabling Constraint Analysis:** The solver can answer critical questions, such as:
  - Are all constraints satisfiable?
  - Are there unreachable options that need adjustment?
  - Are any constraints redundant and can be optimized?
- **Scalability and Efficiency:** As the number of variables and constraints grows, manually checking compatibility quickly becomes infeasible. The solver efficiently handles large sets of constraints, and as such, remains responsive as the complexity of the configuration increases.

DRAFT

The Stage Mechanism

A key backend concept in visualCaseGen is the Stage Mechanism, which structures the CESM configuration process into consecutive steps (stages). Each stage includes a set of related configuration variables that can be adjusted together. Based on the user's selections, different stages are activated dynamically, guiding the user through a structured workflow. The stage mechanism allows the constraint solver to operate incrementally, applying only the relevant constraints for the current stage, which improves performance and responsiveness.

Stage Pipeline

All possible stage paths collectively form the stage pipeline as shown in Figure 2, which dictates the sequence in which configuration variables are presented to the user, and the *precedence of variables* where earlier stages have higher priority over later ones. A complexity arises when the same variable appears in multiple stages. This is allowed as long as it is not reachable along the same path within the stage pipeline. To prevent cyclic dependencies, the stage pipeline must therefore form a directed acyclic graph (DAG), enabling a consistent variable precedence hierarchy and eliminating the possibility of loops or contradictory variable settings.

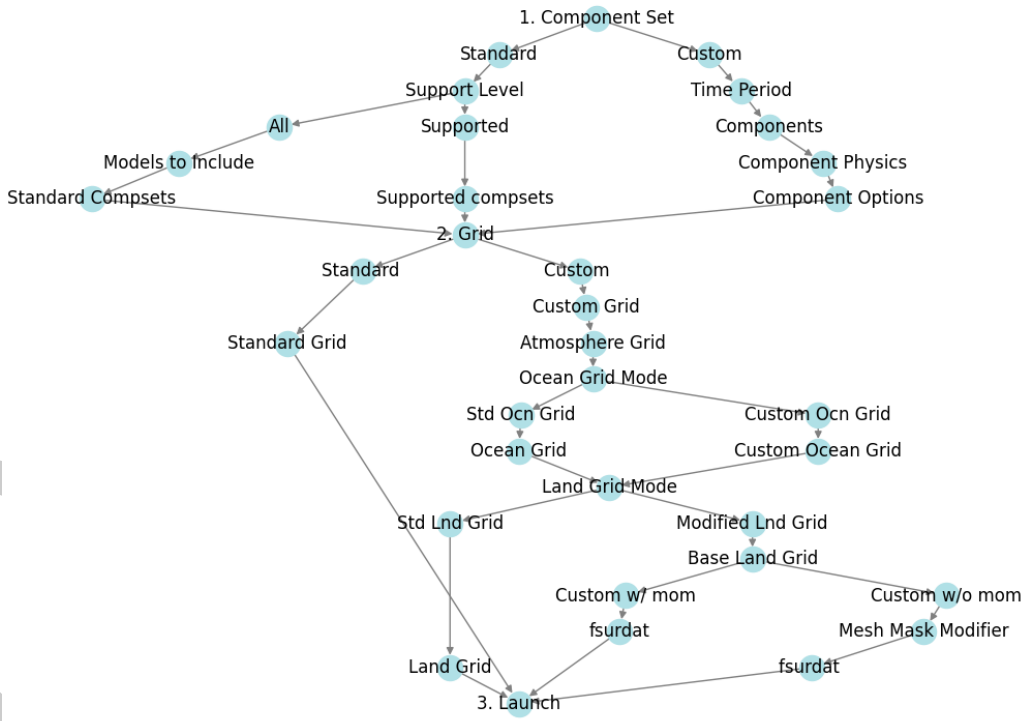

Figure 2: The stage pipeline, starting with the "Component Set" stage and ending with the "Launch" stage. The user follows a path along this pipeline based on their modeling needs and selections.

Constraint Graph and its Traversal

Using the stage pipeline and specified constraints, visualCaseGen constructs a constraint graph, as shown in Figure 3. In this graph:

- Nodes represent configuration variables.
- Directed edges represent dependencies or constraints between variables.
- Edges are directed from higher-precedence variables to lower-precedence variables.

The Journal of Open Source Software

During the configuration process, when a user makes a selection, the constraint graph is traversed to identify all variables that are affected by the selection. This traversal is done in a breadth-first manner, starting from the selected variable and following the edges in the direction of the constraints. The traversal stops at variables whose options' validities are not affected by the selection. As such the traversal is limited to the variables that are directly or indirectly affected by the user's selection, which in turn depends on the the user input, stage hierarchy, and the specified constraints. By dynamically re-evaluating constraints and adjusting available options, visualCaseGen provides real-time feedback, preventing invalid configurations and ensuring scientific consistency in CESM setups.

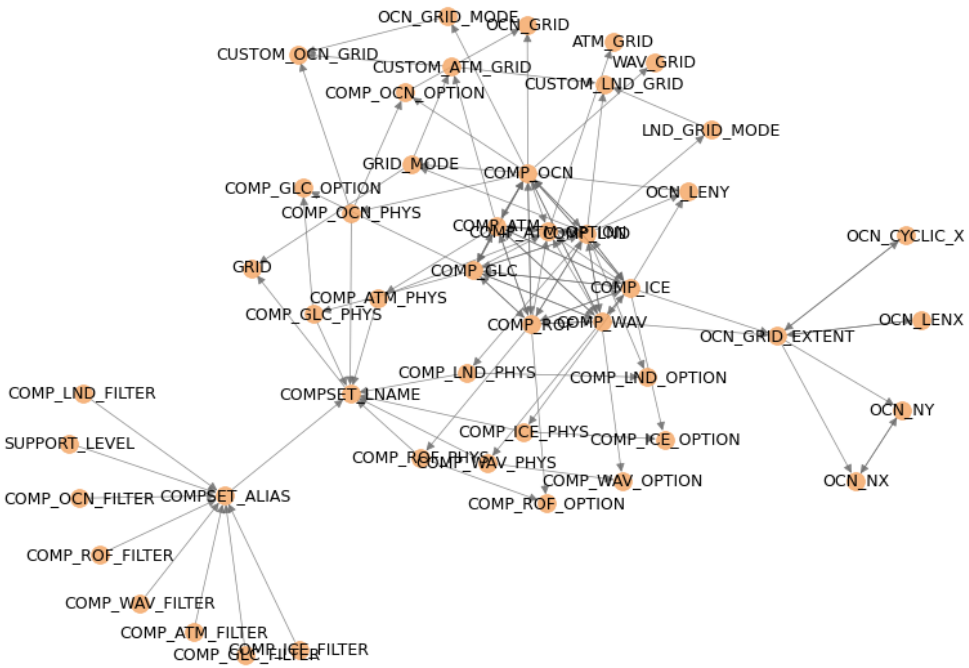

Figure 3: The visualCaseGen constraint graph.

Frontend

The visualCaseGen frontend provides an intuitive and interactive interface for configuring CESM experiments. Built with Jupyter ipywidgets (Jupyter, 2015), it can operate on local machines, HPC clusters, and cloud environments. This portability and flexibility allows researchers to configure CESM experiments efficiently, whether prototyping lightweight simulations on personal computers or running sophisticated applications on remote supercomputing systems.

Figure 4 displays an example stage from the visualCaseGen GUI, where users can select the individual models to be coupled in their CESM experiment. As the user makes selections, the GUI dynamically updates available options by crossing out incompatible choices, ensuring that only valid configurations are selectable. This interactive feedback guides the user toward compatible model setups.

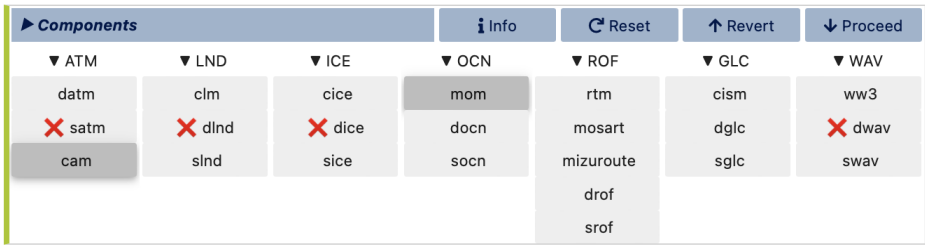

Figure 4: The "Components" stage.

At any stage, users can click on crossed-out options to view a brief explanation of why a particular choice is incompatible with their current selections, as illustrated in Figure 5. This helps guide users through complex dependencies, and helping them make informed adjustments.

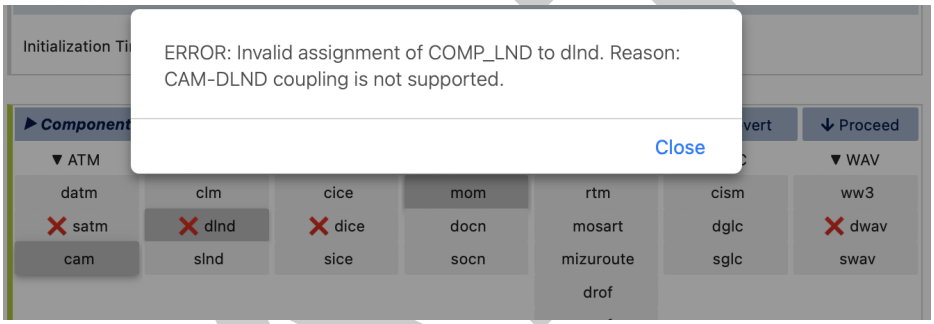

Figure 5: Interactive feedback in incompatible choices.

As another example of streamlining model customization, Figure 6 shows the TopoEditor widget that comes with visualCaseGen. This tool allows users to interactively modify ocean bathymetry, enhancing customizability and ease of use.

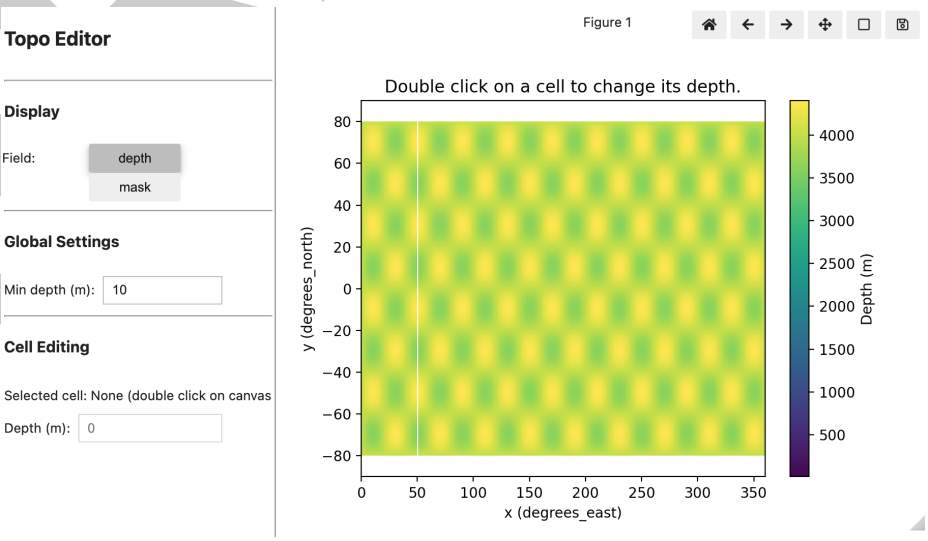

Figure 6: TopoEditor widget

These and similar frontend features enhance accessibility and usability by providing real-time feedback, clarifying dependencies, and helping users understand how their selections shape the experiment configuration.

DRAFT

Remarks

visualCaseGen can significantly accelerate CESM experiment setup for a wide range of studies by automating many aspects of experiment configuration. Instead of manual intervention, modelers can use visualCaseGen's interactive GUI to define model setups, mix and match component configurations, generate custom grids and parameters. The SMT-based constraint solver ensures that only valid model settings are selected, thereby reducing the need for trial-and-error debugging. While complex custom cases may still require fine-tuning, visualCaseGen allows modelers to generate an initial working configuration in a matter of hours rather than weeks, greatly improving efficiency and ease-of-use.

By automating tedious configuration tasks, visualCaseGen enables researchers to focus on scientific exploration rather than technical setup, making CESM more accessible for both idealized and complex modeling studies. To our knowledge, visualCaseGen is the first tool of its kind to integrate SMT-based constraint solving into Earth system model experiment setup, and it exemplifies how formal methods can simplify and enhance the configuration process.

Acknowledgements

This work was supported by the NSF Cyberinfrastructure for Sustained Scientific Innovation (CSSI) program under award number 2004575. Special thanks to the CESM Software Engineering Working Group for their support and feedback during the development of visualCaseGen.

The NSF National Center for Atmospheric Research (NCAR) is a major facility sponsored by the NSF under Cooperative Agreement No. 1852977.

References

Biere, A., Heule, M., & Maaren, H. van. (2009). *Handbook of satisfiability* (Vol. 185). IOS press.

Danabasoglu, G., Lamarque, J.-F., Bacmeister, J., Bailey, D., DuVivier, A., Edwards, J., Emmons, L., Fasullo, J., Garcia, R., Gettelman, A., & others. (2020). The community earth system model version 2 (CESM2). *Journal of Advances in Modeling Earth Systems*, *12*(2), e2019MS001916.

De Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 337–340.

De Moura, L., & Bjørner, N. (2011). Satisfiability modulo theories: Introduction and applications. *Communications of the ACM*, *54*(9), 69–77.

Jupyter. (2015). Ipywidgets: Interactive HTML widgets for jupyter notebooks. In *GitHub repository*. https://github.com/jupyter-widgets/ipywidgets; GitHub.

Maher, P., Gerber, E. P., Medeiros, B., Merlis, T. M., Sherwood, S., Sheshadri, A., Sobel, A. H., Vallis, G. K., Voigt, A., & Zurita-Gotor, P. (2019). Model hierarchies for understanding atmospheric circulation. *Reviews of Geophysics*, *57*(2), 250–280.

Polvani, L., Clement, A., Medeiros, B., Benedict, J., & Simpson, I. (2017). When less is more: Opening the door to simpler climate models, eos, 98. *Eos, Transactions American Geophysical Union*, *99*(3), 15–16.

Wu, X., Reed, K. A., Wolfe, C. L., Marques, G. M., Bachman, S. D., & Bryan, F. O. (2021). Coupled aqua and ridge planets in the community earth system model. *Journal of Advances in Modeling Earth Systems*, *13*(4), e2020MS002418.

DRAFT